



ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ
ГОРОДА МОСКВЫ
Государственное бюджетное профессиональное
образовательное учреждение города Москвы
«Колледж малого бизнеса № 4»
(ГБПОУ КМБ № 4)

GIT

Специальность: 09.02.07 Информационные системы и программирование

Форма обучения: очная

Студент(ка): Белов Кирилл

Группа: ИПО 21.24

Руководитель: Александр Сергеевич Рыбаков

Отчётная работа защищена с оценкой «___» _____

Москва, 2025 г.

Оглавление

Лабораторная 2	4
Лабораторная 3	10
Самостоятельная работа:	17
.....	21
Лабораторная 4	21
Самостоятельная работа.....	25
.....	25
Лабораторная 5	30
Лабораторная 6	32
Лабораторная 7	37

Лабораторная 1

Скачиваем это приложение



Как пользоваться системой гит?

Линукс команды

Ls - для вывода команды

Cd - для перемещения

Pwd – вывод директории

Git bash - консольный интерфейс взаимодействующих с линукс ядром

Повседневные команды:

Git init – инициализация репозитория

Git status – просмотр текущего статуса репозитория

Global – для настройки общих параметров

Git add – добавить

Git commit - - для создания коммита (-m далее пишем текст нашего коммита в “”)

Git commit – попадаю в текстовый редактор vim

Esc -> :q

->: wq – Сохранить файл и выйти

->:x – Выйти без сохранения

Git log - просмотр всех коммитов

Git reset HEAD – возвращает в состояние до индексации

Git checkout “name file” - возвращаем в состояние до изменений (поэтапно)

Git reset –hard HEAD – откат до коммита

Cat – для того , чтобы смотреть

git pull

git push

Git Remote -v – команда для отвязки от одного репозитория к другому

КОНТРОЛЬНЫЕ ВОПРОСЫ 1:

1. СКВ - это Система Контроля Версий, программа для отслеживания истории изменений в файлах проекта.
2. Git решает проблемы: истории изменений, отмены изменений, лёгкости экспериментирования с кодом и совместной работы нескольких разработчиков.
3. Git изначально создан для разработки ядра операционной системы Linux.
4. Репозиторий - это папка с файлами проекта, где Git хранит не только сами файлы, но и всю историю их изменений.
5. Разница: GoogleDrive/DropBox/YandexDisk синхронизируют текущие версии файлов, а Git хранит полную историю всех изменений с возможностью отката к любому моменту.

ТЕРМИНЫ:

Система контроля версий (СКВ) - программа для отслеживания истории изменений в файлах

Git - распределённая система контроля версий, созданная Линусом Торвальдсом

Распределённая система контроля версий - архитектура, где нет центрального сервера, все репозитории равноправны

Репозиторий - папка с файлами проекта, где Git хранит историю изменений и служебную информацию

История изменений - хронологическая запись всех модификаций файлов в проекте

Откат изменений - возврат к предыдущему состоянию файлов

Синхронизация - процесс согласования изменений между разными копиями репозитория
Изоляция изменений - разделение модификаций разных пользователей до момента слияния

Слияние изменений - объединение разных версий файлов в одну

Лабораторная 2

Создаем папку tmp

Имя	Дата изменения	Тип	Размер
 tmp	15.01.2026 11:35	Папка с файлами	

Вход в папку:

```
User@S_BMESH_22 MINGW64 ~  
$ cd C:/D/tmp
```

Регистрируемся:

```
User@S_BMESH_22 MINGW64 /c/D/tmp
$ git config --global user.name "kirill"

User@S_BMESH_22 MINGW64 /c/D/tmp
$ git config --global user.email "kirillbelov326@gmail.com"
```

```
User@S_BMESH_22 MINGW64 /c/D/tmp
$ ls

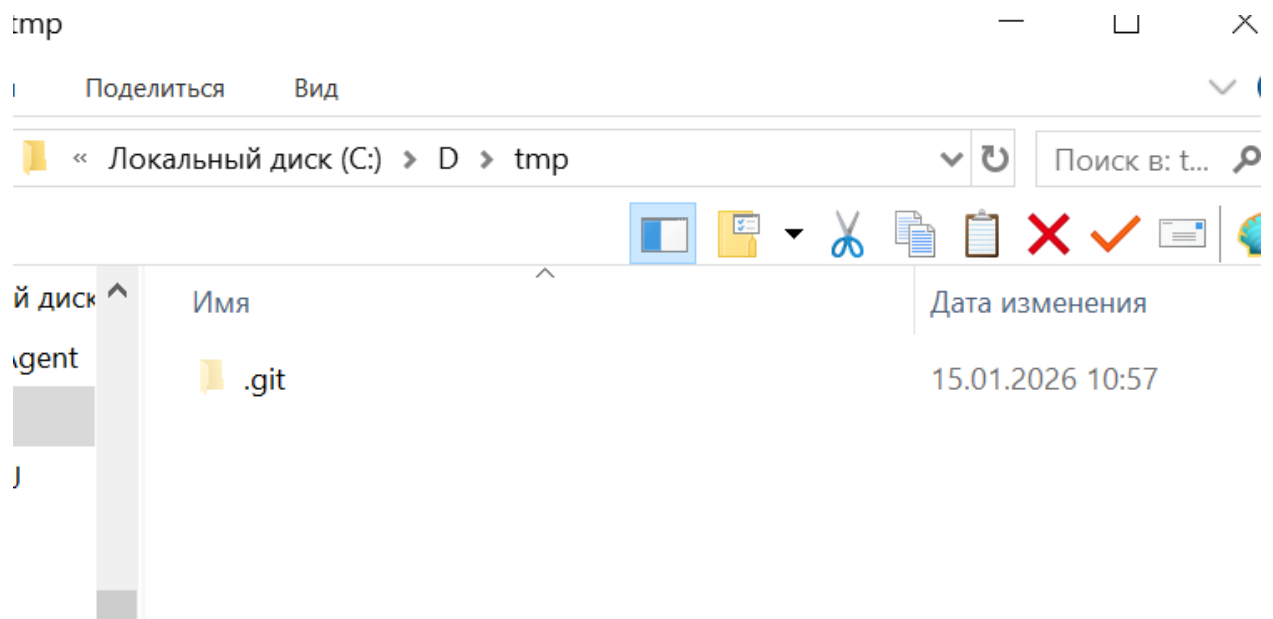
User@S_BMESH_22 MINGW64 /c/D/tmp
$ git init
Initialized empty Git repository in C:/D/tmp/.git/

User@S_BMESH_22 MINGW64 /c/D/tmp (master)
$ ls

User@S_BMESH_22 MINGW64 /c/D/tmp (master)
$ ls -a
./  ../  .git/

User@S_BMESH_22 MINGW64 /c/D/tmp (master)
$
```

В папке появилось в «скрытых элементов» папка Git



Создаем в папке D папку GitRepo переносим в папку tmp

```
User@S_BMESH_22 MINGW64 /c/D/tmp
$ cd C:/D/GitRepo/tmp

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp (master)
$ git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp (master)
$ |
```

КОНТРОЛЬНЫЕ ВОПРОСЫ 2:

1 GitBash - это командная строка для Windows, предоставляющая окружение Linux для работы с Git.

2 Команда ls показывает список файлов и папок в текущей директории.

3 Сменить директорию можно командой cd (change directory).

4 Текущую директорию отображает команда pwd (print working directory).

5 Настройки Git задаются для отслеживания авторства изменений (имя и email).

6 Для всех репозиториев настройки применяются с ключом --global.

ТЕРМИНЫ:

git init - команда для создания нового репозитория Git

git status - команда для проверки состояния репозитория

Консольный клиент Git - текстовый интерфейс для работы с Git через командную строку

Графический клиент Git - программа с графическим интерфейсом для работы с Git

Инициализация репозитория - процесс создания служебных файлов Git в папке проекта

Скрытые папки (скрытые файлы) - служебные директории Git, начинающиеся с точки (.git)

Состояние репозитория - информация о текущих изменениях, готовых к коммиту

Настройки авторства - конфигурация имени и email пользователя для фиксации в коммитах

Глобальные настройки (--global) - конфигурация, применяемая ко всем репозиториям на компьютере

Локальные настройки (--local) - конфигурация, действующая только в текущем репозитории

`git config --global user.name "Имя"` - установка глобального имени пользователя

`git config --global user.email "email"` - установка глобального email • `git config --local` - установка настроек только для текущего репозитория

`git init` - создание нового репозитория в текущей папке • `git status` - проверка состояния репозитория

`ls` - вывод списка файлов и папок

`ls -a` - вывод списка всех файлов, включая скрытые

cd [путь] - переход в указанную директорию

pwd - вывод текущей директории

Лабораторная 3

Создаем в папке tmp папку README.md и прописываем эти команды:

```
fatal: pathspec 'README.md' did not match any files

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  README.md.txt
```

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp (master)
$ git add README.md

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp (master)
$
```

Добавляем файл README

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp (master)
$ git add .

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp (master)
$ git commit -m "Добавляем файл README"
On branch master
nothing to commit, working tree clean

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp (master)
$ git status
On branch master
nothing to commit, working tree clean

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp (master)
$
```

Открываем редми и пишем hello word

Проверка README

```
On branch master
nothing to commit, working tree clean

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp (master)
$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   README.md

no changes added to commit (use "git add" and/or "git commit -a")

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp (master)
```

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp (master)
$ git add README.md

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp (master)
```

[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Wiki](#) [Security](#) [Insights](#) [Settings](#)

 **GitTest** Public Pin Watch 0

 main

 1 Branch

 0 Tags

Go to file

+

Code

 **kirillbelov67** Initial commit e53443a · 3 minutes ago 1 Commit

 README.md

 Initial commit 3 minutes ago

README 

GitTest

MINGW64:/c/D/GitRepo/tmp

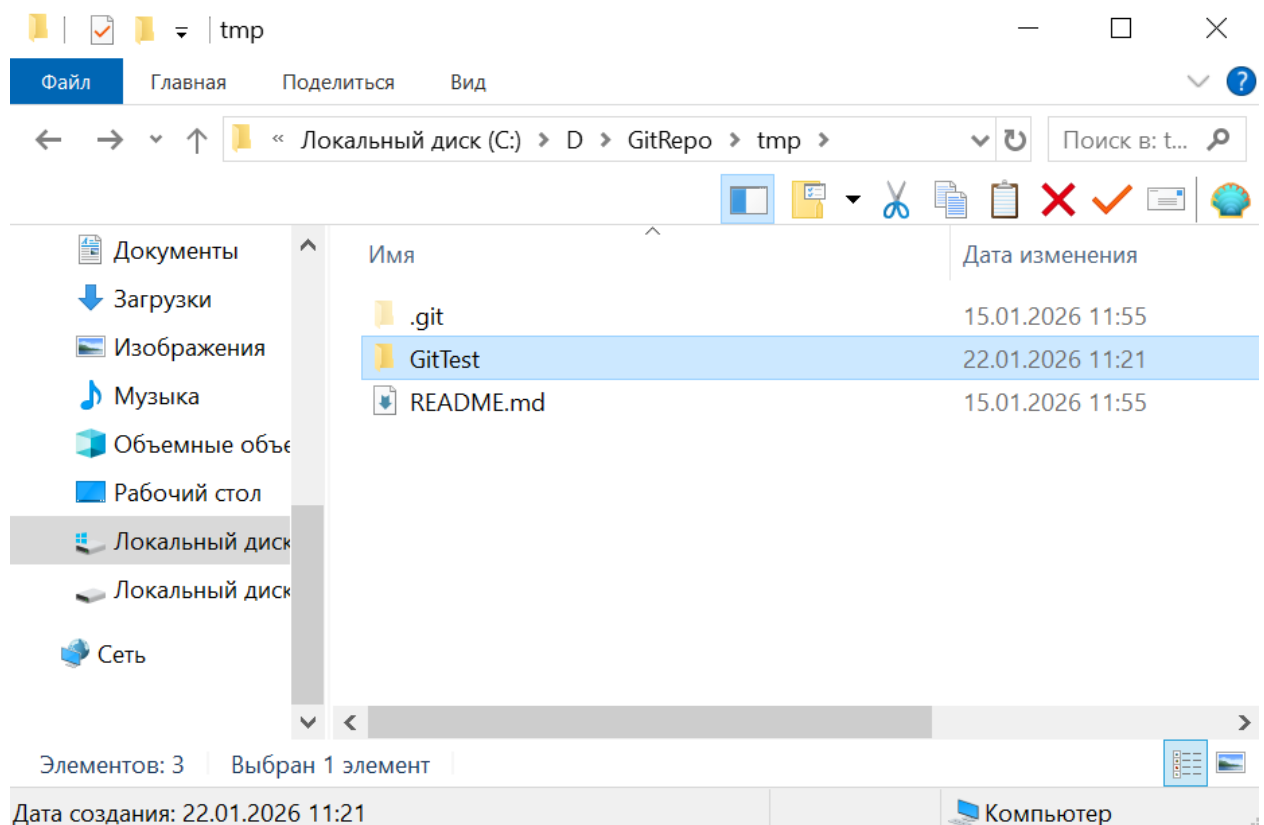
he most similar command is
stash

```
ser@S_BMESH_22 MINGW64 ~  
git clone https://github.com/kirillbelov67/GitTest.git  
Cloning into 'GitTest'...  
remote: Enumerating objects: 3, done.  
remote: Counting objects: 100% (3/3), done.  
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)  
Receiving objects: 100% (3/3), done.
```

```
ser@S_BMESH_22 MINGW64 ~  
cd C:/D/GitRepo/tmp
```

```
ser@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp (master)  
git clone https://github.com/kirillbelov67/GitTest.git  
Cloning into 'GitTest'...  
remote: Enumerating objects: 3, done.  
remote: Counting objects: 100% (3/3), done.  
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)  
Receiving objects: 100% (3/3), done.
```

```
ser@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp (master)
```



Receiving objects: 100% (3/3), done.

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp (master)
\$ cd GitTest

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/GitTest (main)
\$ git add hi.txt

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/GitTest (main)
\$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
 (use "git restore --staged <file>..." to unstage)
 new file: hi.txt

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/GitTest (main)
\$ |

Your branch is up to date with 'origin/main'.

Changes to be committed:

(use "git restore --staged <file>..." to unstage)
new file: hi.txt

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/GitTest (main)

\$ git log

commit e53443a6104effe9cd5b0834e4f5d5f4ed1f3960 (HEAD -> main, origin/main)
Author: kirillbelov67 <kirillbelov326@gmail.com>
Date: Thu Jan 22 11:16:00 2026 +0300

Initial commit

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/GitTest (main)

\$ git commit/

git: 'commit/' is not a git command. See 'git --help'.

The most similar command is
commit

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/GitTest (main)

\$ git commit -m "hi"

[main cf71755] hi

1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 hi.txt

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/GitTest (main)

\$ git status

On branch main

Your branch is ahead of 'origin/main' by 1 commit.

(use "git push" to publish your local commits)

nothing to commit, working tree clean

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/GitTest (main)

\$ git push

Enumerating objects: 4, done.

Counting objects: 100% (4/4), done.

Delta compression using up to 4 threads

Compressing objects: 100% (2/2), done.

Writing objects: 100% (3/3), 266 bytes | 88.00 KiB/s, done.

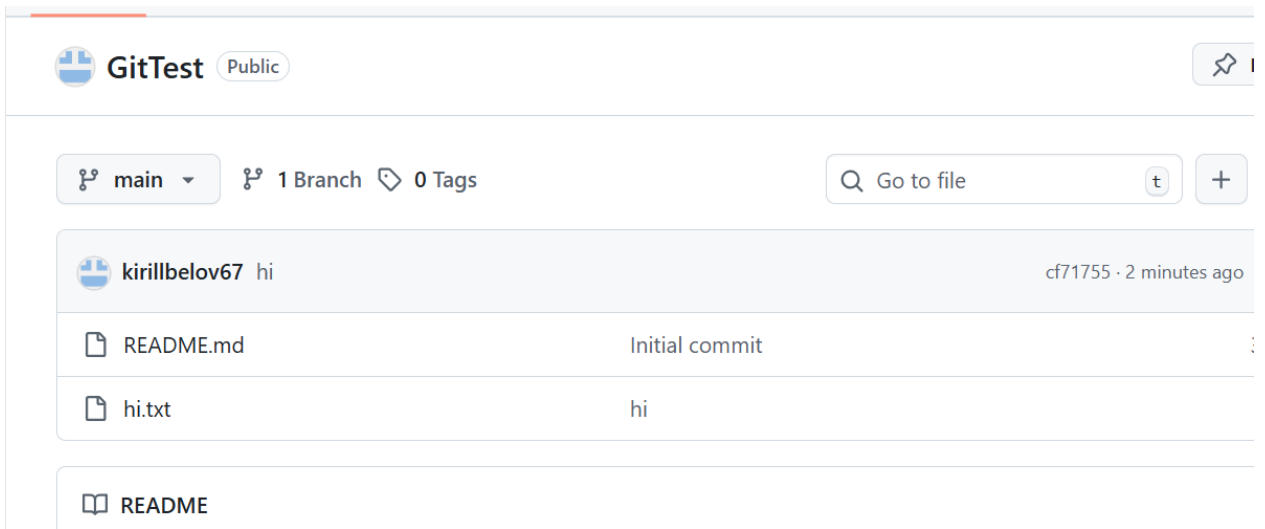
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)

To https://github.com/kirillbelov67/GitTest.git

e53443a..cf71755 main -> main

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/GitTest (main)

\$



```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/GitTest (main)
$ git pull
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 918 bytes | 41.00 KiB/s, done.
From https://github.com/kirillbelov67/GitTest
   cf71755..1749019  main       -> origin/main
Updating cf71755..1749019
Fast-forward
 README.md | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/GitTest (main)
$ git log
commit 1749019bb2c108e2fda4cce1886c876f1d92fe83 (HEAD -> main, origin/main, origin/HEAD)
Author: kirillbelov67 <kirillbelov326@gmail.com>
Date:   Thu Jan 22 11:52:33 2026 +0300

    Update README.md

commit cf717550eab88f54003252e4bdf82cd45268be6f
Author: kirill <kirillbelov326@gmail.com>
Date:   Thu Jan 22 11:48:36 2026 +0300

    hi

commit e53443a6104effe9cd5b0834e4f5d5f4ed1f3960
Author: kirillbelov67 <kirillbelov326@gmail.com>
Date:   Thu Jan 22 11:16:00 2026 +0300

    Initial commit

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/GitTest (main)
$ |
```



```
Initial commit

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/GitTest (main)
$ cat README.md
пр

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/GitTest (main)
$ git remote -v
origin https://github.com/kirillbelov67/GitTest.git (fetch)
origin https://github.com/kirillbelov67/GitTest.git (push)

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/GitTest (main)
$ |
```

Самостоятельная работа:

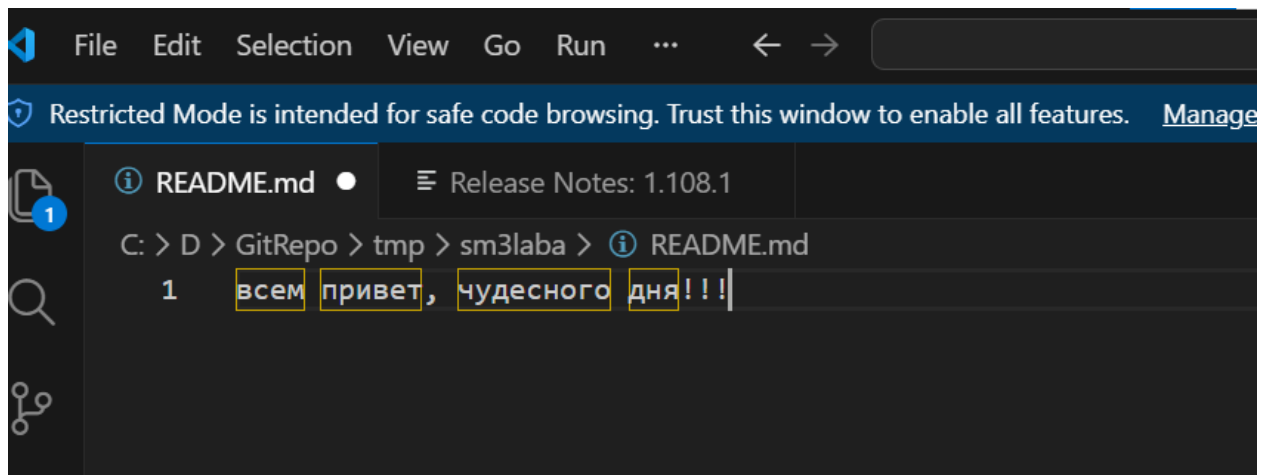
Создаем папку для репозитория, переходим в эту папку через команду `cd` и инициализируем репозиторий.

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp (master)
$ cd C:/D/GitRepo/tmp/sm3laba

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ git init
Initialized empty Git repository in C:/D/GitRepo/tmp/sm3laba/.git/

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$
```

Добавляем в папку файл `README.md` и пишем в нем «всем привет, чудесного дня!!!»



Выполняем команду `git status` для просмотра текущего статуса проекта, видим что файл находится в неотслеживаемом состоянии, для того чтобы он изменил свое состояние до отслеживаемого изменяемого выполняем команду `git add *`. Сохраняем изменения через команду `git commit -m "README.md"`

📁 minirepo /c/d/GitRepo/tmp/sm3laba

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md

nothing added to commit but untracked files present (use "git add" to track)

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ git *
git: 'README.md' is not a git command. See 'git --help'.

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ git add *

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   README.md

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ git commit -m "README.md"
error: pathspec '-m' did not match any file(s) known to git
error: pathspec '"README.md"' did not match any file(s) known to git

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ git commit -m "README"
error: pathspec '-m' did not match any file(s) known to git
error: pathspec '"README"' did not match any file(s) known to git

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ git commit -m "README"
[master (root-commit) e09bde8] README
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 README.md

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$
```

КОММИТИМ ПО ЗАДАНИЮ

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ git commit -m "добавление 3 файлов"
[master 41a02e4] добавление 3 файлов
3 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 1.txt
create mode 100644 2.txt
create mode 100644 3.txt
```

```

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ git add *

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ git status
bash: git: command not found

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   1.txt
        new file:   2.txt
        new file:   3.txt

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$

```

После выполнения всех коммитов просматриваем список этих коммитов через команды `git log`

```

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ git log
commit 41a02e47a281d81d1cca1b884021a08406c2ef9c (HEAD -> master)
Author: kirill <kirillbelov326@gmail.com>
Date: Thu Jan 29 09:45:59 2026 +0300

    добавление 3 файлов

commit e09bde830d3031f284961d3db56c5f60bdfd08ae
Author: kirill <kirillbelov326@gmail.com>
Date: Thu Jan 29 09:26:26 2026 +0300

    README

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$

```

Через команду `mkdir` создаем подпапку и добавляем в нее файлы

```

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ mkdir подпапка

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ git add подпапка

```

```

nothing added to commit but untracked files present (use 'git add' to track)
User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ git add подпапка

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ git commit -m "добавление файлов в подпапку"
[master 820d176] добавление файлов в подпапку
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 "\320\277\320\276\320\264\320\277\320\260\320\277\320\272\320\260\1.txt"

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ |

```

Через команду `rm подпапка/*` удаляем все файлы в подпапке

```

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ rm подпапка/*

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ git add *

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    "\320\277\320\276\320\264\320\277\320\260\320\277\320\272\320\260\1.txt"

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ |

```

Через команду `rmdir подпапка` удаляем подпапку

```

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ rmdir подпапка

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ git add
Nothing specified, nothing added.
hint: Maybe you wanted to say 'git add .'
hint: Disable this message with "git config set advice.addEmptyPathsSpec false"

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba (master)
$ git add *

```

КОНТРОЛЬНЫЕ ВОПРОСЫ 3:

1. `git status`
2. Файлы, которые находятся в папке репозитория, но не отслеживаются Git
3. `git add [имя_файла]`
4. `git log`
5. `git reset`, не рекомендуется потому что это нарушает историю изменений

6. Сравнивая текущие файлы с сохранёнными копиями в папке .git

ТЕРМИНЫ:

Неотслеживаемые файлы (Untracked files) - файлы в папке репозитория, которые Git видит, но не контролирует

Подготовленные файлы (Staged files) - файлы, готовые к фиксации изменений после команды add

Неизменённые файлы (Unmodified files) - файлы, не менявшиеся со времени последнего коммита

Изменённые файлы (Modified files) - файлы, которые были изменены после последнего коммита

Индексация (стейджинг) - процесс подготовки файлов к коммиту с помощью git add

Фиксация изменений (коммит) - сохранение текущего состояния файлов в истории

Git Лог изменений - история всех коммитов в репозитории

Хеш коммита - уникальный идентификатор коммита из шестнадцатеричных цифр
Откат коммитов - возврат к предыдущему состоянию репозитория

HEAD - указатель на текущий коммит в репозитории

Отмена индексации - возврат файлов из состояния staged в modified

Контентно-адресуемая файловая система - внутренняя архитектура хранения данных в Git
git add [файл] - добавить файл под контроль Git или

проиндексировать изменения git commit -m "сообщение" - зафиксировать

изменения с комментарием git log - показать историю коммитов git log -p -

показать историю с подробными изменениями в файлах git rm [файл] -

удалить файл из индекса и рабочей папки git rm --cached [файл] - удалить

файл только из индекса

Git `git reset HEAD` - откат к последнему коммиту (отмена индексации) `git reset --hard [коммит]` - полный откат к указанному коммиту с потерей всех изменений `git checkout -- [файл]` - отмена изменений в файле до состояния последнего коммита

Лабораторная 4

Создаем репозиторий на компьютере и инициализируем его. Связываем его с первым репозиторием и проверяем. Скачиваем данные из гитхаба.

```

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba/GitTest (main)
$ cd C:/D/GitRepo/tmp/sm3laba/GitTest

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba/GitTest (main)
$ cd GitTest
bash: cd: GitTest: No such file or directory

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/sm3laba/GitTest (main)
$ cd C:/D/GitRepo/tmp/GitTest

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/GitTest (main)
$ cd GitTest
bash: cd: GitTest: No such file or directory

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp/GitTest (main)
$ cd C:/D/GitRepo/sm4laba

User@S_BMESH_22 MINGW64 /c/D/GitRepo/sm4laba
$ git init
Initialized empty Git repository in C:/D/GitRepo/sm4laba/.git/

User@S_BMESH_22 MINGW64 /c/D/GitRepo/sm4laba (master)
$ git remote add origin https://github.com/kirillbelov67/GitTest.git

User@S_BMESH_22 MINGW64 /c/D/GitRepo/sm4laba (master)
$

User@S_BMESH_22 MINGW64 /c/D/GitRepo/sm4laba (master)
$ git remote -v
origin https://github.com/kirillbelov67/GitTest.git (fetch)
origin https://github.com/kirillbelov67/GitTest.git (push)

User@S_BMESH_22 MINGW64 /c/D/GitRepo/sm4laba (master)
$ git fetch origin
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 9 (delta 0), reused 3 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (9/9), 1.95 KiB | 14.00 KiB/s, done.
From https://github.com/kirillbelov67/GitTest
* [new branch]      main      -> origin/main

```

Создаем репозиторий А в гитхабе и добавлем в качестве origin для первого репозитория , предварительно удалив предыдущий origin

```

User@S_BMESH_22 MINGW64 /c/D/GitRepo/sm4laba (master)
$ git remote remove origin

User@S_BMESH_22 MINGW64 /c/D/GitRepo/sm4laba (master)
$ git remote add origin https://github.com/kirillbelov67/A.git

```

Отправляем данные в репозиторий А и клонируем его себе

Самостоятельная работа

Создаем репозиторий на компьютере и инициализируем его

```
User@S_BMESH_22 MINGW64 ~  
$ cd /c/D/GitRepo  
  
User@S_BMESH_22 MINGW64 /c/D/GitRepo  
$ mkdir Fish2_228  
  
User@S_BMESH_22 MINGW64 /c/D/GitRepo  
$ cd /c/D/GitRepo/Fish2_228  
  
User@S_BMESH_22 MINGW64 /c/D/GitRepo/Fish2_228  
$ git init  
Initialized empty Git repository in C:/D/GitRepo/Fish2_228/.git/
```

Связываем его с первым репозиторием и проверяем

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/Fish2_228 (master)  
$ git remote add origin https://github.com/kirillbelov67/Fisch2_228.git  
  
User@S_BMESH_22 MINGW64 /c/D/GitRepo/Fish2_228 (master)  
$ git remote -v  
origin https://github.com/kirillbelov67/Fisch2_228.git (fetch)  
origin https://github.com/kirillbelov67/Fisch2_228.git (push)  
  
User@S_BMESH_22 MINGW64 /c/D/GitRepo/Fish2_228 (master)  
$
```

Скачиваем данные из гитхаба

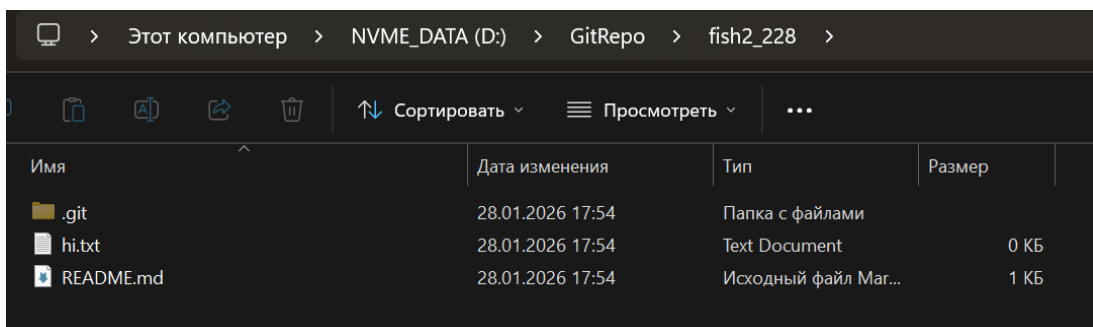
```

User@S_BMESH_22 MINGW64 /c/D/GitRepo/Fish2_228 (master)
$ git fetch origin
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 858 bytes | 7.00 KiB/s, done.
From https://github.com/kirillbelov67/Fisch2_228
* [new branch]      main      -> origin/main

User@S_BMESH_22 MINGW64 /c/D/GitRepo/Fish2_228 (master)
$ |

```

И видим что все данные из первого репозитория перенеслись во второй



Создаем репозиторий А в гитхабе и добавлем в качестве origin для первого репозитория , предварительно удалив предыдущий origin

```

User@S_BMESH_22 MINGW64 /c/D/GitRepo/Fish2_228 (master)
$ git remote remove origin

User@S_BMESH_22 MINGW64 /c/D/GitRepo/Fish2_228 (master)
$ git remote add origin https://github.com/kirillbelov67/A..git

User@S_BMESH_22 MINGW64 /c/D/GitRepo/Fish2_228 (master)

```

Отправляем данные в репозиторий А и клонируем его себе

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/Fish2_228 (master)
$ git push -u origin main
error: src refspec main does not match any
error: failed to push some refs to 'https://github.com/kirillbelov67/A..git'

User@S_BMESH_22 MINGW64 /c/D/GitRepo/Fish2_228 (master)
$ |
```

Отправляем данные в репозитории А и клонируем его себе

Что не работает

КОНТРОЛЬНЫЕ ВОПРОСЫ 4:

1. git push
2. git pull
3. git commit -m "сообщение"
4. Для хостинга проектов, коллективной разработки, синхронизации между компьютерами
5. Разработчики клонируют общий репозиторий, работают в своих ветках, создают пул-реквесты для слияния изменений

ТЕРМИНЫ:

Хостинг IT-проектов - веб-сервис для размещения и совместной разработки репозитория

Клонирование репозитория - создание локальной копии удалённого репозитория

Удалённый репозиторий (remote) - репозиторий, размещённый на сервере или другом компьютере

Локальный репозиторий - репозиторий на вашем компьютере

Origin - стандартное имя для удалённого репозитория-источника

Синхронизация репозитория - процесс обмена изменениями между локальным и удалённым репозиториями

Отправка изменений (push) - передача локальных коммитов в удалённый репозиторий

Получение изменений (pull) - загрузка изменений из удалённого репозитория в локальный

Приватный репозиторий - репозиторий с ограниченным доступом, видный только владельцу

Публичный репозиторий - репозиторий, доступный для просмотра всем пользователям

Форк (fork) - создание копии чужого репозитория в своём аккаунте

Пул-реквест (pull request) - предложение изменений автору исходного репозитория

Ветвление от чужого репозитория - создание собственной версии проекта на основе существующего

Тарифный план GitHub - различные уровни доступа к функциям сервиса

Связь между репозиториями - настройка удалённых источников для обмена изменениями

Бесплатный аккаунт GitHub - учётная запись с базовыми функциями для публичных репозиториях

Платные возможности GitHub - дополнительные функции вроде приватных репозиториев

Удалённый оригинал - основной репозиторий, с которого производилось клонирование `git clone [url] [папка]` - клонировать удалённый репозиторий в указанную папку `git remote -v` - показать список удалённых репозиториев `git push` - отправить локальные изменения в удалённый репозиторий `git pull` - получить изменения из удалённого репозитория и слить с локальным `git fetch` - получить изменения из удалённого репозитория без слияния `git remote add [имя] [url]` - добавить удалённый репозиторий

Лабораторная 5

Создаем папку и прописываем

```
cd
User@S_BMESH_22 MINGW64 ~
$ cd C:/D/GitRepo/tmp2
User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp2
```

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo2
$ git init
Initialized empty Git repository in C:/D/GitRepo2/.git/
```

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo2 (master)
$ git add .

User@S_BMESH_22 MINGW64 /c/D/GitRepo2 (master)
$ git commit -m "Init"
[master (root-commit) 9772b5c] Init
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 README.md

User@S_BMESH_22 MINGW64 /c/D/GitRepo2 (master)
```

Initialized empty Git repository in C:/D/GitRepo/tmp

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp2 (master)
$ git remote add hihik C:/D/GitRepo2
```

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp2 (master)
$
```

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp2 (master)
$ git remote
hihik

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp2 (master)
$ git remote -v
hihik    C:/D/GitRepo2 (fetch)
hihik    C:/D/GitRepo2 (push)

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp2 (master)
$ git fetch hihik
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 188 bytes | 5.00 KiB/s, done.
From C:/D/GitRepo2
 * [new branch]      master    -> hihik/master
```

```
* [new branch]      master    -> hihik/master

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp2 (master)
$ git checkout --track hihik/master
branch 'master' set up to track 'hihik/master'.
Already on 'master'

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp2 (master)
$ |
```

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp2 (master)
$ git pull
Already up to date.

User@S_BMESH_22 MINGW64 /c/D/GitRepo/tmp2 (master)
$
```

КОНТРОЛЬНЫЕ ВОПРОСЫ 5:

1. Ветка - последовательность коммитов в истории изменений
2. Локальные (на компьютере) и удалённые (на сервере или другом компьютере)
3. `git remote add [имя] [путь]`
4. `git remote -v`
5. Показывает подробную информацию об удалённом репозитории

ТЕРМИНЫ:

Удалённые ветки - ссылки на состояние веток в удалённых репозиториях

Локальные ветки - ветки, существующие только в вашем локальном репозитории

Распределённая система контроля версий - архитектура, где все репозитории равноправны

Связь между репозиториями - подключение локального репозитория к удалённому

Отслеживание удалённой ветки - связывание локальной ветки с соответствующей удалённой

Получение информации об изменениях (`fetch`) - загрузка данных об изменениях без их применения

Адрес удалённого репозитория - путь или URL для подключения к другому репозиторию

Список удалённых репозиторийев - перечень всех подключённых внешних источников

Равноправные репозитории - все репозитории в Git имеют одинаковый статус

Связь веток между репозиториями - сопоставление локальных и удалённых веток

Имя удалённого репозитория - алиас для обращения к подключённому внешнему репозиторию

Перенос информации об изменениях - копирование метаданных коммитов между репозиториями

Выкачивание изменений - процесс загрузки коммитов из удалённого репозитория

Отслеживаемая ветка - локальная ветка, связанная с удалённой для синхронизации

git remote - показать список подключённых удалённых репозиторийев git remote add [имя] [путь] - добавить удалённый репозиторий git remote -v - показать подробный список удалённых репозиторийев с адресами git remote show [имя] - показать детальную информацию об удалённом репозитории git fetch [имя] - получить информацию об изменениях из удалённого репозитория git checkout --track [удалённый/ветка] - создать локальную ветку для отслеживания удалённой git pull - получить изменения из удалённого репозитория и применить их

Лабораторная 6

создаем репозитории, добавлем текст документ и созданием коммитов

```

User@S_BMESH_22 MINGW64 /c/D/GitRepo2 (master)
$ cd C:/D/GitRepo/branches

User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches
$ git init
Initialized empty Git repository in C:/D/GitRepo/branches/.git/

User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ git add .

User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ git commit -m "np"
[master (root-commit) 86a2af0] np
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 hi.txt

User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ git tag 1.0

User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ git tag
1.0

User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ git show 1.0
commit 86a2af071d4a87ae10021ed8c2ba65b4027069a6 (HEAD -> master, tag: 1.0)
Author: kirill <kirillbelov326@gmail.com>
Date: Thu Jan 29 11:47:29 2026 +0300

    np

diff --git a/hi.txt b/hi.txt
new file mode 100644
index 0000000..e69de29

```

```

User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ git tag -l "1.*"
1.0

User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ |

```

MINGW64:/c/D/GitRepo/branches

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ git tag -d 1.0
Deleted tag '1.0' (was 86a2af0)
```

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
```

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ git tag -a 1.0 -m "test tag"
```

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ git show 1.0
tag 1.0
Tagger: kirill <kirillbelov326@gmail.com>
Date: Thu Feb 5 10:43:40 2026 +0300

test tag

commit 86a2af071d4a87ae10021ed8c2ba65b4027069a6 (HEAD -> master, tag: 1.0)
Author: kirill <kirillbelov326@gmail.com>
Date: Thu Jan 29 11:47:29 2026 +0300

    np

diff --git a/hi.txt b/hi.txt
new file mode 100644
index 0000000..e69de29

User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ |
```

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ git log --oneline
86a2af0 (HEAD -> master, tag: 1.0) np
```

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ git tag 1.1 86a2af0

User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$
```

 Public repository

 main ▾

Code ▾

...

 **kirillbelov67** 1 minute ago

...



 README.md

1 minute ago

 **README**



```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ git submodule add https://github.com/kirillbelov67/git2.git
Cloning into 'C:/D/GitRepo/branches/git2'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
warning: in the working copy of '.gitmodules', LF will be replaced by CRLF the next time Git touches it

User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ |
```

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ ls
git2/  hi.txt

User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   .gitmodules
    new file:   git2

User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
```

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ cat .gitmodules
[submodule "git2"]
    path = git2
    url = https://github.com/kirillbelov67/git2.git

User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ git add .

User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ |
```

```
User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ git commit -m "new submodule"
[master 5e17da2] new submodule
 2 files changed, 4 insertions(+)
 create mode 100644 .gitmodules
 create mode 160000 git2

User@S_BMESH_22 MINGW64 /c/D/GitRepo/branches (master)
$ |
```

КОНТРОЛЬНЫЕ ВОПРОСЫ 6:

1. Метка (tag) - пометка определённого коммита как важного момента в истории
2. Легковесные метки и аннотированные метки
3. `git tag [имя_метки]` для легковесной или `git tag -a [имя] -m "сообщение"` для аннотированной
4. `git show [имя_метки]`
5. Для подключения одного репозитория как подмодуля внутри другого репозитория
6. `git submodule add [URL] [папка]`

ТЕРМИНЫ:

Легковесные метки - простые указатели на определённый коммит без дополнительной информации

Аннотированные метки - полноценные объекты Git с контрольной суммой, автором, датой и сообщением

Маска для меток - шаблон для фильтрации меток по имени

Цифровая подпись метки - криптографическое подтверждение авторства с помощью GPG Релиз-мастер - разработчик, отвечающий за выпуск версий продукта

Субмодуль - репозиторий внутри другого репозитория

Подключение библиотеки через субмодуль - включение внешнего кода как зависимого репозитория

Файл `.gitmodules` - конфигурационный файл, описывающий все подключённые субмодули

Инициализация субмодулей - подготовка субмодулей после клонирования репозитория
Обновление субмодулей - загрузка файлов из подключённых субмодулей

Метки версий - теги для отметки релизов (v1.0, v2.0 и т.д.)

Автоматические системы сборки - инструменты, реагирующие на теги для запуска сборки продукта

Пометка готового к сборке продукта - использование аннотированных тегов для релизов
Визу на публикацию - утверждение коммита для выпуска в продакшн

Передача меток в удалённый репозиторий - отправка тегов на сервер командой push

Маркировка важных коммитов - использование тегов для обозначения ключевых моментов разработки
git tag - показать список меток git tag [имя] - создать легковесную метку git tag -a [имя] -m "сообщение" - создать аннотированную метку git tag -l 'шаблон' - показать метки по шаблону git tag -d [имя] - удалить метку git show [метка] - показать информацию о метке git push origin [метка] - отправить конкретную метку в удалённый репозиторий git push origin --tags - отправить все метки в удалённый репозиторий
git submodule add [url] [папка] - добавить субмодуль git submodule init - инициализировать субмодули git submodule update - обновить субмодули

Лабораторная 7

Создаем папку для репозитория

```
g
User@S_BMESH_22 MINGW64 ~
$ cd C:/D/ProGit/branches

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches
$ git init
Initialized empty Git repository in C:/D/ProGit/branches/.git/
```

Проверяем есть ли у нас коммиты

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

Создаем readme и добавляем ее в контроль гит

Проверяем статус репозитория и в какой ветке находимся

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ git add .

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ git commit -m "first commit"
[master (root-commit) 98cb0e2] first commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 readme.md

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ git log
commit 98cb0e26e1dcb9c2eb71dbb55610d91c81d3d794 (HEAD -> master)
Author: kirill <kirillbelov326@gmail.com>
Date: Mon Feb 9 13:23:38 2026 +0300

    first commit

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
```

Создаем новую ветку


```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ git branch testing

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ git status
On branch master
nothing to commit, working tree clean
```

переходим в эту ветку

```
nothing to commit, working tree clean

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ git checkout testing
Switched to branch 'testing'

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (testing)
$ git status
On branch testing
nothing to commit, working tree clean
```

Меняем файл README и коммитим в ветке testing Проверяем коммиты и видим что коммиты находятся в разных ветках

```
MINGW64:/c/D/ProGit/branches
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (testing)
$ git add .

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (testing)
$ git commit -m "КОММИТ В testing"
[testing 1b9d9ae] КОММИТ В testing
1 file changed, 1 insertion(+)

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (testing)
$ git log
commit 1b9d9ae117a2a53114b9e6222d865c644c87ae7b (HEAD -> testing)
Author: kirill <kirillbelov326@gmail.com>
Date: Mon Feb 9 13:35:35 2026 +0300

    КОММИТ В testing

commit 98cb0e26e1dcb9c2eb71dbb55610d91c81d3d794 (master)
Author: kirill <kirillbelov326@gmail.com>
Date: Mon Feb 9 13:23:38 2026 +0300

    first commit

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (testing)
$ |
```

Переходим в ветку мастер и смотрим коммиты

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (testing)
$ git checkout master
Switched to branch 'master'

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ git log
commit 98cb0e26e1dcb9c2eb71dbb55610d91c81d3d794 (HEAD -> master)
Author: kirill <kirillbelov326@gmail.com>
Date: Mon Feb 9 13:23:38 2026 +0300

    first commit
```

Добавляем новый файл и передаем под контроль гита в ветке master

```
FIRST COMMIT

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ git add .

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$
```

Коммитим и смотрим список коммитов, видим что добавился новый коммит

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ git commit -m "добавление index.txt в ветке master"
[master f43c9ac] добавление index.txt в ветке master
2 files changed, 1 insertion(+)
create mode 100644 first.txt
create mode 100644 index.txt

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ git log
commit f43c9aca7e850f88db7d713bb45746e8a5423b36 (HEAD -> master)
Author: kirill <kirillbelov326@gmail.com>
Date: Mon Feb 9 13:55:48 2026 +0300

    добавление index.txt в ветке master

commit 98cb0e26e1dcb9c2eb71dbb55610d91c81d3d794
Author: kirill <kirillbelov326@gmail.com>
Date: Mon Feb 9 13:23:38 2026 +0300

    first commit

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
```

Переходим в ветку testing и видим что ничего не поменялось в этой ветке, так и осталось 2 коммита

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ git checkout testing
Switched to branch 'testing'

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (testing)
$ git log
commit 1b9d9ae117a2a53114b9e6222d865c644c87ae7b (HEAD -> testing)
Author: kirill <kirillbelov326@gmail.com>
Date: Mon Feb 9 13:35:35 2026 +0300

    коммит в testing

commit 98cb0e26e1dcb9c2eb71dbb55610d91c81d3d794
Author: kirill <kirillbelov326@gmail.com>
Date: Mon Feb 9 13:23:38 2026 +0300

    first commit

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (testing)
$
```

Создаем новую ветку перед этим перейдя в ветку master

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (testing)
$ git checkout master
Switched to branch 'master'

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ git branch hotfix
```

переходим в эту ветку

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ git branch hotfix

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ git checkout hotfix
Switched to branch 'hotfix'
```

Меняем файл index и смотрим статус

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (hotfix)
$ git status
On branch hotfix
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.txt

no changes added to commit (use "git add" and/or "git commit -a")

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (hotfix)
$
```

Добавляем изменения под контроль гита и коммитим

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (hotfix)
$ git add .

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (hotfix)
$ git commit -m "добавили красную кнопку"
[hotfix 6d47bb9] добавили красную кнопку
1 file changed, 1 insertion(+)
```

Список коммитов и видим что в нашей новой ветке сохранились все коммиты из master и новый наш

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (hotfix)
$ git log
commit 6d47bb9a21976cfa179834097758765791ef6f8a (HEAD -> hotfix)
Author: kirill <kirillbelov326@gmail.com>
Date: Mon Feb 9 14:04:00 2026 +0300

    добавили красную кнопку

commit f43c9aca7e850f88db7d713bb45746e8a5423b36 (master)
Author: kirill <kirillbelov326@gmail.com>
Date: Mon Feb 9 13:55:48 2026 +0300

    добавление index.txt в ветке master

commit 98cb0e26e1dcb9c2eb71dbb55610d91c81d3d794
Author: kirill <kirillbelov326@gmail.com>
Date: Mon Feb 9 13:23:38 2026 +0300

    first commit

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (hotfix)
$
```

Переходим в ветку Master и выводим список коммитов, видим что прошлый коммит который был создан в новой ветке не отображается

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (hotfix)
$ git checkout master
Switched to branch 'master'

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ git log
commit f43c9aca7e850f88db7d713bb45746e8a5423b36 (HEAD -> master)
Author: kirill <kirillbelov326@gmail.com>
Date: Mon Feb 9 13:55:48 2026 +0300

    добавление index.txt в ветке master

commit 98cb0e26e1dcb9c2eb71dbb55610d91c81d3d794
Author: kirill <kirillbelov326@gmail.com>
Date: Mon Feb 9 13:23:38 2026 +0300

    first commit

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
```

Объединяем с доп веткой

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ git merge hotfix
Updating f43c9ac..6d47bb9
Fast-forward
 index.txt | 1 +
 1 file changed, 1 insertion(+)

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ |
```

Переходим в папку и смотрим что индекс изменился, а до этого он был без дополнительной надписи которую мы коммитили в другой ветке с которой объединили

Смотрим список коммитов и видим что теперь 3 коммита и один из них после объединения двух веток

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ git log
commit 6d47bb9a21976cfa179834097758765791ef6f8a (HEAD -> master, hotfix)
Author: kirill <kirillbelov326@gmail.com>
Date:   Mon Feb 9 14:04:00 2026 +0300

    добавили красную кнопку

commit f43c9aca7e850f88db7d713bb45746e8a5423b36
Author: kirill <kirillbelov326@gmail.com>
Date:   Mon Feb 9 13:55:48 2026 +0300

    добавление index.txt в ветке master

commit 98cb0e26e1dcb9c2eb71dbb55610d91c81d3d794
Author: kirill <kirillbelov326@gmail.com>
Date:   Mon Feb 9 13:23:38 2026 +0300

    first commit

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$
```

Переходим в ветку testing и просматриваем коммиты

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ git checkout testing
Switched to branch 'testing'

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (testing)
$ git log
commit 1b9d9ae117a2a53114b9e6222d865c644c87ae7b (HEAD -> testing)
Author: kirill <kirillbelov326@gmail.com>
Date: Mon Feb 9 13:35:35 2026 +0300

    КОММИТ В testing

commit 98cb0e26e1dcb9c2eb71dbb55610d91c81d3d794
Author: kirill <kirillbelov326@gmail.com>
Date: Mon Feb 9 13:23:38 2026 +0300

    first commit

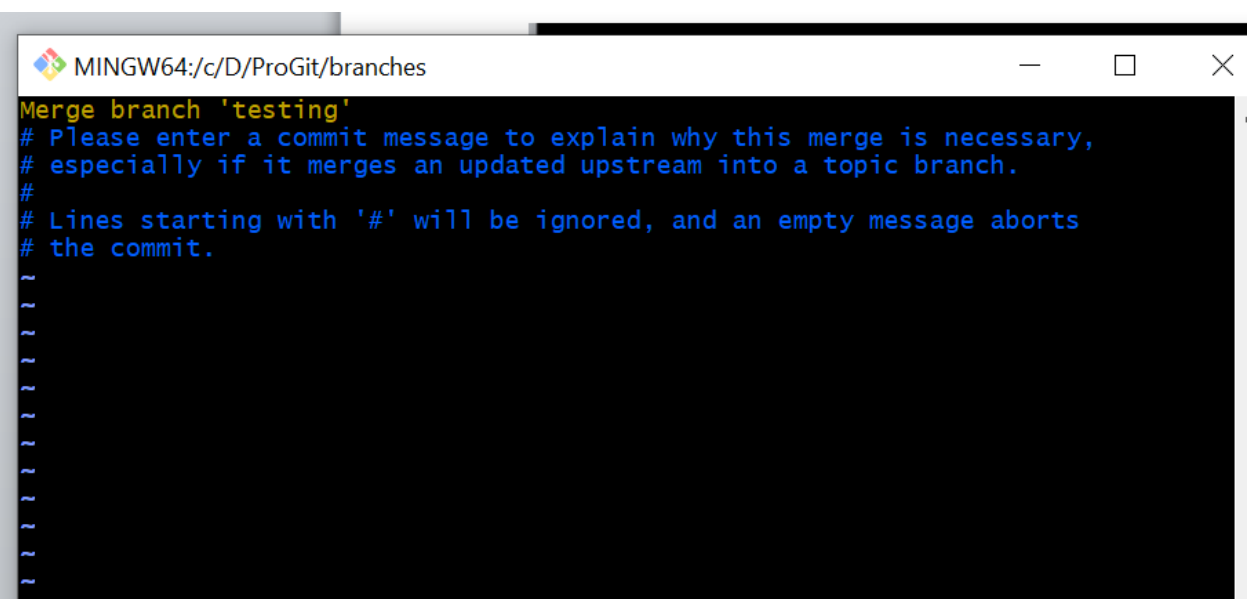
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (testing)
```

Переходим обратно в ветку master и сливаем с веткой testing

```
User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (testing)
$ git checkout master
Switched to branch 'master'

User@S_BMESH_06 MINGW64 /c/D/ProGit/branches (master)
$ git merge testing|
```

открывается текстовый редактор



```
MINGW64:/c/D/ProGit/branches
Merge branch 'testing'
# Please enter a commit message to explain why this merge is necessary,
# especially if it merges an updated upstream into a topic branch.
#
# Lines starting with '#' will be ignored, and an empty message aborts
# the commit.
~
~
~
~
~
~
~
~
```

Вернулись обратно в git bash через последовательное нажатие <esc> : q

```
Merge made by the 'ort' strategy.  
readme.md | 1 +  
1 file changed, 1 insertion(+)  
  
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)  
$
```

Вводим команду git log и видим созданный коммит который мы могли переименовать в текстовом редакторе но мы этого не делали

```
commit 6d47bb9a21976cfa179834097758765791ef6f8a (hotfix)  
Author: kirill <kirillbelov326@gmail.com>  
Date: Mon Feb 9 14:04:00 2026 +0300  
  
добавили красную кнопку
```

Коммитим

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)  
$ git add .  
  
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)  
$ git commit -m "создали файл"  
[master efb5ac0] создали файл  
1 file changed, 1 deletion(-)  
  
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
```

Создаем ветку и переходим в нее

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)  
$ git checkout -b feature/1  
Switched to a new branch 'feature/1'  
  
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (feature/1)  
$
```

Мы поменяли текстовый файл добавили новую надпись и закоммитили изменения


```

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (feature/1)
$ git add .

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (feature/1)
$ git commit -m "добавили красную кнопку"
[feature/1 9bb0eb7] добавили красную кнопку
1 file changed, 1 insertion(+)

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (feature/1)
$

```

Перешли обратно в ветку мастер и создали вторую ветку и перешли в нее

```

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (feature/1)
$ git checkout master
Switched to branch 'master'

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ git branch feature/2

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (master)
$ git checkout feature/2
Switched to branch 'feature/2'

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (feature/2)
$ |

```

Добавили надпись две зеленых кнопки и закоммитили изменения

```

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (feature/2)
$ git add .

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (feature/2)
$ git commit -m "две зеленых кнопки"
On branch feature/2
nothing to commit, working tree clean

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (feature/2)
$

```

Переходим в первую ветку где красная кнопка объединяем ее со второю где зеленые

```

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (feature/2)
$ git checkout feature/1
Switched to branch 'feature/1'

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (feature/1)
$ git merge feature/2
Auto-merging first.txt
CONFLICT (content): Merge conflict in first.txt
Automatic merge failed; fix conflicts and then commit the result.

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (feature/1|MERGING)
$

```

Мы видим надпись CONFLICT – в файле first.txt возник мергеконфликт. Автоматическое слияние невозможно. Нужно поправить конфликты и закоммитить результат.

Заходим в файл текстовый и видим

```

Файл  Правка  Формат  Вид  Справк
|<<<<<< HEAD
|екрцкер
|=====
|ffffff
|>>>>>> feature/2

```

Удаляем все служебные команды и сохраняем файл смотрим статус репозитория

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (feature/1|MERGING)
$ git status
On branch feature/1
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   first.txt

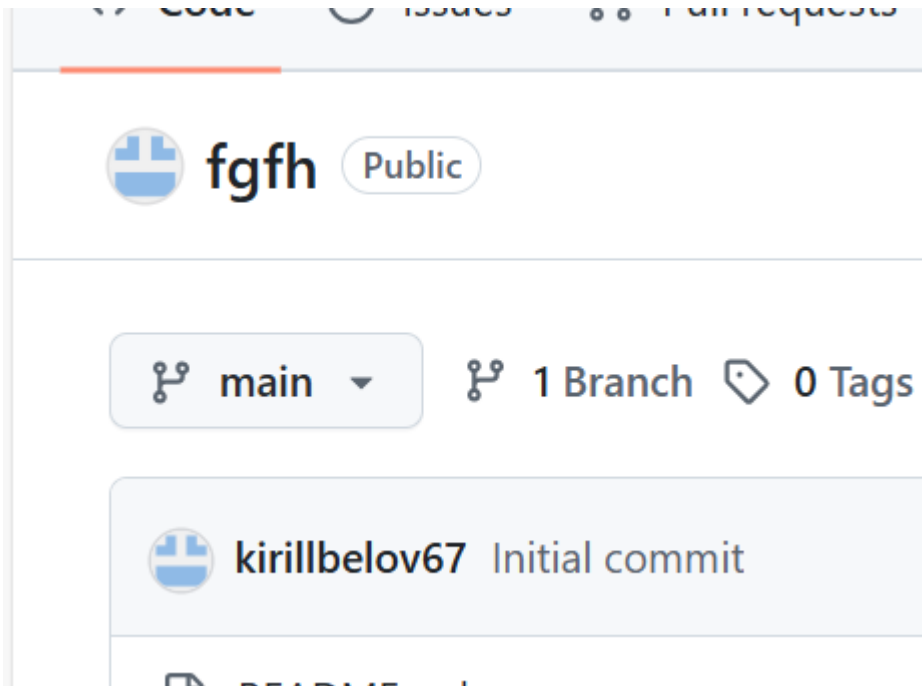
no changes added to commit (use "git add" and/or "git commit -a")
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (feature/1|MERGING)
$
```

Коммитим изменения

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (feature/1|MERGING)
$ git commit -m "конфликт решен"
[feature/1 2dc5b64] конфликт решен
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (feature/1)
$
```

Самостоятельная работа

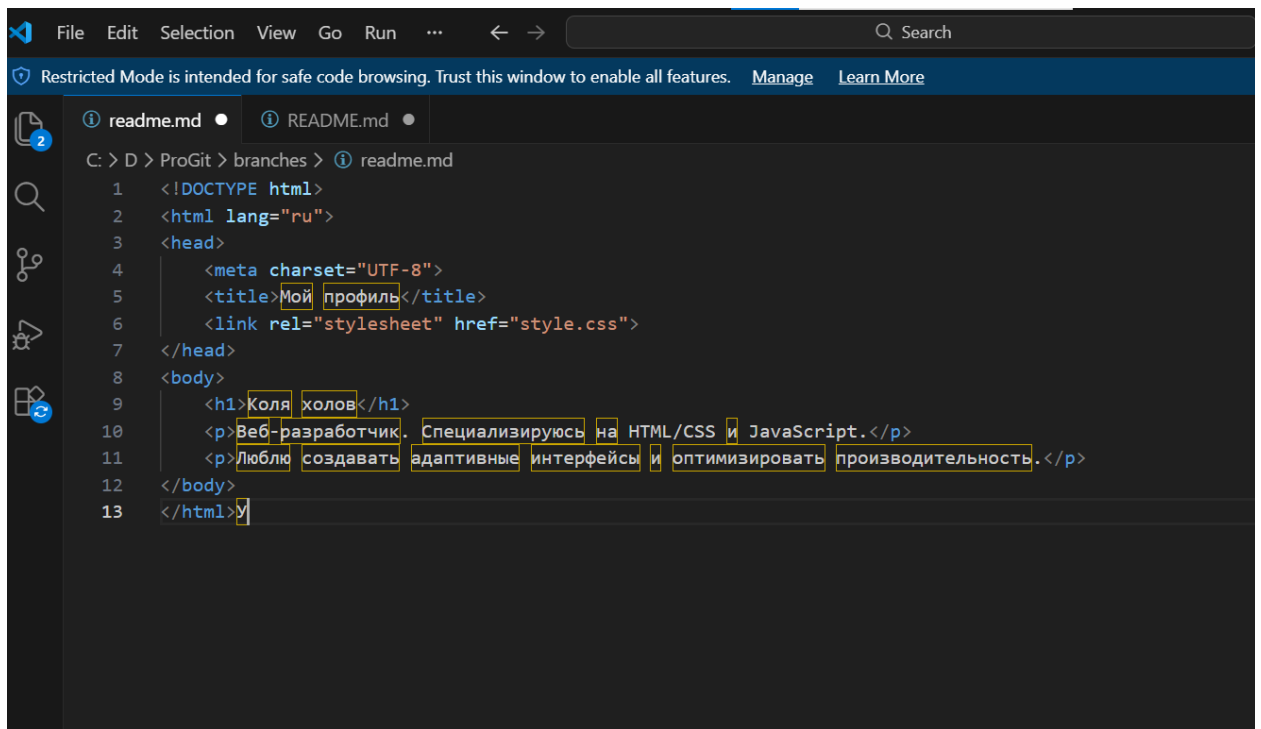
Создаем новый пустой репозиторий на гитхабе



Клонируем его в папку D вместо диска D

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (feature/1)
$ git clone https://github.com/kirillbelov67/fgfh.git
Cloning into 'fgfh'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (3/3), done.
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (feature/1)
```

Созадем файл HTML с моим профилем



Переходим в папку репозитория

Добавляем добавленный файл по контроль гита

И коммитим изменения

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (feature/1)
$ cd C:/D/fgfh
bash: cd: C:/D/fgfh: No such file or directory

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches (feature/1)
$ cd C:/D/ProGit/branches/fgfh

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
$ git add .
```

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
$ git commit -m "добавил index.html с базовой информацией"
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  index.html

nothing added to commit but untracked files present (use "git add" to track)

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
```

Пушим в гитхаб в ветку мейн

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
$ git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 333 bytes | 333.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/kirillbelov67/fgfh.git
0ec1c98..bb3dc74  main -> main

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
```

добавляем ветку и сразу же переходим в нее

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
$ git checkout -b add-subscribe-button
Switched to a new branch 'add-subscribe-button'
```

Добавляем кнопку

```
<p> я хочу отчислиться.</p>
<button> хочу отчислиться </button>
</body>
```

Добавляем изменения под контроль гит и коммитим

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (add-subscribe-button)
$ git commit -m "добавлена кнопка 'хочу отчислиться'"
On branch add-subscribe-button
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (add-subscribe-button)
$ |
```

Пушим на гит хаб через ветку созданную для кнопки

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (add-subscribe-button)
$ git push origin add-subscribe-button
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'add-subscribe-button' on GitHub by visiting:
remote:   https://github.com/kirillbelov67/fgfh/pull/new/add-subscribe-button
remote:
To https://github.com/kirillbelov67/fgfh.git
 * [new branch]      add-subscribe-button -> add-subscribe-button

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (add-subscribe-button)
$ |
```

Переходим в ветку main

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (add-subscribe-button)
$ git checkout main
M       index.html
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
```

Выполняем слияние веток

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
$ git merge add-subscribe-button
Updating bb3dc74..a35eb23
Fast-forward
 index.html | 15 +++++
 1 file changed, 15 insertions(+)
```

И пушим на гитхаб

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
$ git push origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/kirillbelov67/fgfh.git
bb3dc74..a35eb23  main -> main

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
```

Создаем ветку для изменения цвета кнопки

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
$ git checkout -b change-button-color
Switched to a new branch 'change-button-color'
```

Изменяем цвет в файле html

```
<button type="button">Подписаться</button>
<button
background-color: #5d6d7e;
color: white;
border: none;

padding: 10px 20px;>Подписаться</button>
body>
```

Добавляем изменения под контроль гит и коммитим

```
nothing to commit, working tree clean

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (change-button-color)
$ git add .

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (change-button-color)
$ git commit -m "изменен цвет кнопки на зеленый"
[change-button-color 60f78d9] изменен цвет кнопки на зеленый
1 file changed, 6 insertions(+)

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (change-button-color)
$ |
```


Пушим на гитхаб

```
1 file changed, 6 insertions(+)

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (change-button-color)
$ git push origin change-button-color
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 465 bytes | 232.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'change-button-color' on GitHub by visiting
remote:   https://github.com/kirillbelov67/fgfh/pull/new/change-button-co
remote:
To https://github.com/kirillbelov67/fgfh.git
 * [new branch]      change-button-color -> change-button-color

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (change-button-color)
```

Переходим в ветку мэйн

Сливаем ветки

И пушим на гитхаб

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (change-button-color)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
$ git merge change-button-color
Updating a35eb23..60f78d9
Fast-forward
 index.html | 6 ++++++
 1 file changed, 6 insertions(+)

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
$ git push origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/kirillbelov67/fgfh.git
 a35eb23..60f78d9  main -> main

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
```

Добавляем ветку

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
$ git checkout -b add-social-login
Switched to a new branch 'add-social-login'

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (add-social-
$
```

Добавляем форму входа через соцсети

```
<div class="social-login">
  <button>Войти через Google</button>
  <button>Войти через VK</button>
</div>
```

Добавляем изменения под контроль ГИТ и коммитим

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (add-social-login)
$ git add .

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (add-social-login)
$ git commit -m"добавлена форма входа через соц-сети"
[add-social-login 1db3b69] добавлена форма входа через соц-сети
1 file changed, 5 insertions(+)

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (add-social-login)
```

Пушим на гитхаб

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (add-social-login)
$ git push origin add-social-login
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 454 bytes | 227.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'add-social-login' on GitHub by visiting:
remote:   https://github.com/kirillbelov67/fgfh/pull/new/add-social-login
remote:
To https://github.com/kirillbelov67/fgfh.git
 * [new branch]      add-social-login -> add-social-login

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (add-social-login)
```

Переходим в ветку main

Сливаем ветки

Пушим

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (add-social-login)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
$ git merge add-social-login
Updating 60f78d9..1db3b69
Fast-forward
 index.html | 5 +++++
 1 file changed, 5 insertions(+)

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
$ git push origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/kirillbelov67/fgfh.git
 60f78d9..1db3b69  main -> main

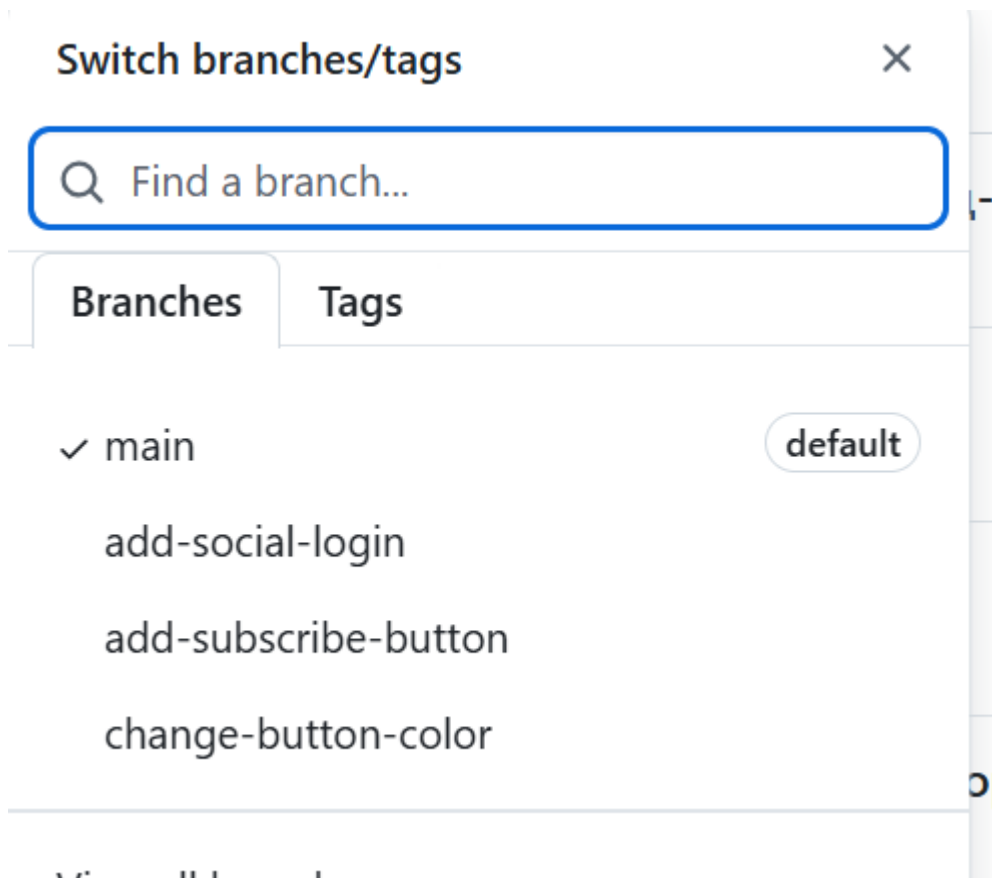
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
```

Проверяем гитхаб Видим что было 5 коммита

Commits on Feb 9, 2026

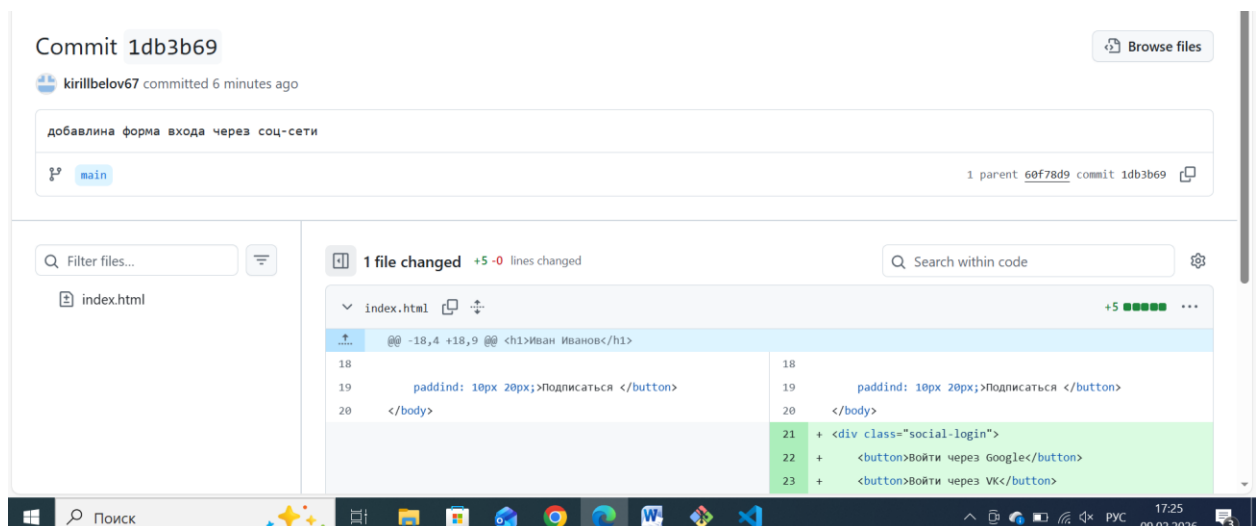
добавлена форма входа через соц-сети kirillbelov67 committed 4 minutes ago	1db3b69		<>
изменен цвет кнопки на зеленый kirillbelov67 committed 11 minutes ago	60f78d9		<>
добавлена кнопка kirillbelov67 committed 18 minutes ago	a35eb23		<>
добавил index.html с базовой информацией kirillbelov67 committed 2 hours ago	bb3dc74		<>
Initial commit kirillbelov67 authored 2 hours ago	0ec1c98		<>

идим что были добавлены ветки



Вывел второй коммит из гит хаба

Видим что произошло слияние и что есть родительский коммит и итоговый номер коммита, также видно что поменялось



Создаем ветку теста для создания конфликта

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgf
$ git checkout -b conflict-test
Switched to a new branch 'conflict-test'
```

Изменяем имя кнопки на «класс!»

```
<button> Класс!</button>
```

Добавляем под контроль гит пушим

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (conflict-test)
$ git add .

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (conflict-test)
$ git commit -m"Изменено название кнопки в conflict-test"
[conflict-test 981b48a] Изменено название кнопки в conflict-test
1 file changed, 1 insertion(+), 1 deletion(-)
```

Переходим в ветку main

```
User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (conflict-test)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

И добавляем изменений в эту же строчку

```
<p> я хочу повесится </p>
<button> Класс</button>
<button>
```

Добавляем изменения под контроль гит и коммитим

Слияем с веткой теста конфликта и видим что возник конфликт

```

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
$ git add .

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
$ git commit -m"Изменено название кнопки в main"
[main 6158b1f] Изменено название кнопки в main
1 file changed, 1 insertion(+), 1 deletion(-)

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
$ git merge conflict-test
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main|MERGING)

```

Заходим в файл индекса и видим то что сконфликтовало

```

2  <p> я хочу повесится </p>
   Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
3  <<<<<< HEAD (Current Change)
4  <button> Клас</button>
5  =====
6  <button> Клас</button>
7  >>>>>> conflict-test (Incoming Change)
8  <button>

```

Выбираем какое в итоге оставляем изменение

И добавляем это под контроль гит и коммитим

```

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main|MERGING)
$ git add .

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main|MERGING)
$ git commit -m"Разрешен конфликт в index.html"
[main f476a23] Разрешен конфликт в index.html

```

Пушим на гитхаб

```

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)
$ git push origin main
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 4 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 1002 bytes | 334.00 KiB/s, done.
Total 9 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 1 local object.
To https://github.com/kirillbelov67/fgfh.git
    1db3b69..f476a23  main -> main

User@S_BMESH_22 MINGW64 /c/D/ProGit/branches/fgfh (main)

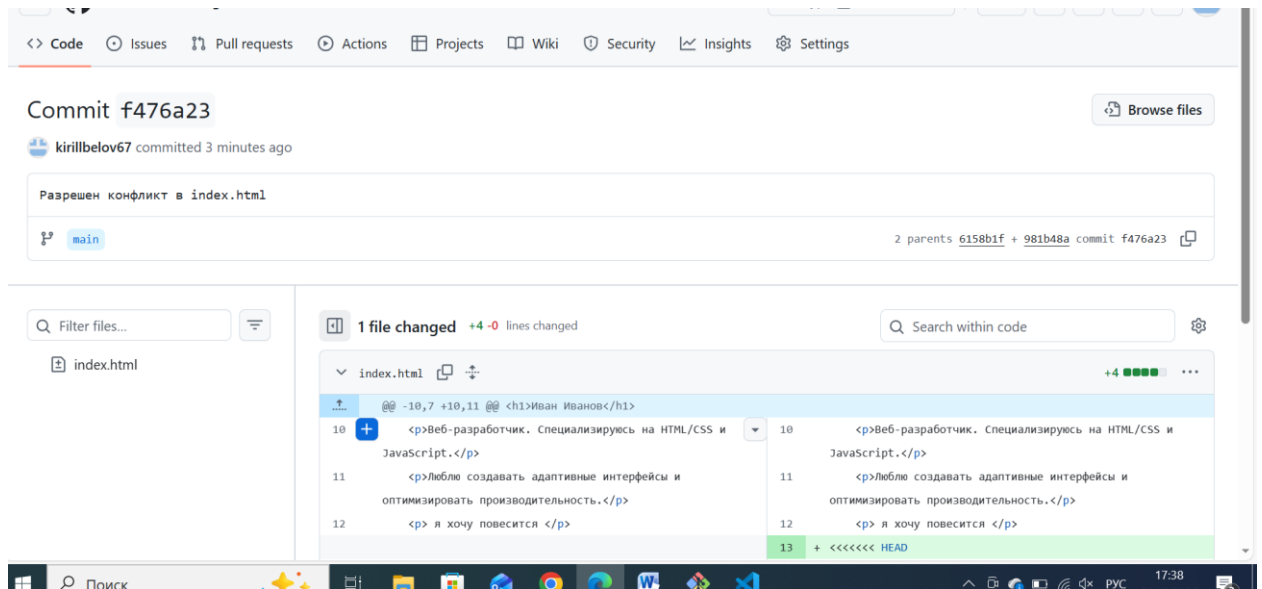
```

Заходим на гит хаб и виидм в коммитах все сохраненные коммиты

Commits on Feb 2, 2020

Разрешен конфликт в index.html kirillbelov67 committed 1 minute ago	f476a23	 
Изменено название кнопки в main kirillbelov67 committed 4 minutes ago	6158b1f	 
Изменено название кнопки в conflict-test kirillbelov67 committed 7 minutes ago	981b48a	 
добавлена форма входа через соц-сети kirillbelov67 committed 18 minutes ago	1db3b69	 
изменен цвет кнопки на зеленый kirillbelov67 committed 24 minutes ago	60f78d9	 
добавлена кнопка kirillbelov67 committed 31 minutes ago	a35eb23	 
добавил index.html с базовой информацией kirillbelov67 committed 2 hours ago	bb3dc74	 
Initial commit kirillbelov67 authored 2 hours ago	0ec1c98	  Verified

Так же видим коммит с разрешением конфликта



Ссылка на гитхаб <https://github.com/kirillbelov67/fgfh.git>

КОНТРОЛЬНЫЕ ВОПРОСЫ 7:

1. master (или main)
2. git branch [имя_ветки]
3. HEAD - указатель на текущую ветку и коммит; git checkout перемещает HEAD
4. git merge [ветка_для_слияния]
5. Быстрое слияние, когда история остаётся линейной

ТЕРМИНЫ:

Снапшот - полный снимок состояния репозитория в определённый момент

Указатель на коммит - ссылка на конкретное состояние в истории

Разветвление истории - расхождение веток от общего предка

Слияние веток (merge) - объединение изменений из разных веток

Fast-forward слияние - линейное слияние, когда ветки находятся на одной линии истории

Алгоритм наилучшего общего предка - метод поиска общего родителя для слияния

Рекурсивная стратегия слияния - алгоритм слияния расходящихся веток

Коммит слияния (merge commit) - специальный коммит с двумя родителями

Конфликт слияния - ситуация, когда изменения в разных ветках затрагивают одни и те же строки

Метки конфликта - специальные маркеры (<<<<<<, =====, >>>>>>) в файле при конфликте

Ручное разрешение конфликтов - редактирование файлов для устранения противоречий

Чистое состояние репозитория - отсутствие незакоммиченных изменений при переключении веток

Указатель на ветку - ссылка на последний коммит в ветке

Общий родитель - коммит, от которого разошлись ветки

Штатная операция - стандартная, часто используемая процедура

Принцип "одна задача - одна ветка" - стратегия изоляции изменений по задачам
Перемотка истории - перемещение указателя ветки вперёд по коммитам

Противоречия при слиянии - несовместимые изменения в одном месте файла

Автоматическое слияние - разрешение конфликтов Git без вмешательства пользователя

Стратегия слияния - алгоритм объединения изменений из разных веток

git branch - показать список веток git branch [имя] - создать новую ветку

git branch -v - показать ветки с последними коммитами git checkout [ветка] -

переключиться на ветку git checkout -b [имя] - создать ветку и переключиться

на неё git merge [ветка] - слить указанную ветку в текущую git branch -d

[ветка] - удалить ветку git stash - временно сохранить незакоммиченные изменения git stash apply - восстановить сохранённые изменения